

CRONOGRAMA COMPLETO DE APRENDIZAJE JAVA, TESTING Y DEVOPS

MÓDULO 1 — Java de 0 a 100 (Aprox 60 clases × 1 hora)

BLOQUE 1: Fundamentos (Clases 1–10)

| Clase | Tema |

|-----|-----|

| 1 | Introducción a Java: historia, JDK vs JRE vs JVM, instalación, primer `Hello World`
|

| 2 | Variables, tipos de datos primitivos (`int`, `double`, `boolean`, `char`,
`String`) |

| 3 | Operadores aritméticos, relacionales y lógicos |

| 4 | Estructuras de control: `if`, `else if`, `else`, operador ternario |

| 5 | Bucles: `for`, `while`, `do-while` |

| 6 | `switch` / `switch` expressions (Java 14+) |

| 7 | Arrays unidimensionales y multidimensionales |

| 8 | Métodos: definición, parámetros, retorno, sobrecarga |

| 9 | Recursividad: factorial, Fibonacci, torres de Hanói |

| 10 | Manejo de entrada/salida: `Scanner`, `System.out` |

BLOQUE 2: Programación Orientada a Objetos (Clases 11–25)

| Clase | Tema |

|-----|-----|

| 11 | Clases y objetos: atributos, métodos, constructores |

| 12 | Encapsulamiento: `private`, `public`, `protected`, getters/setters |

| 13 | Herencia: `extends`, uso de `super` |

| 14 | Polimorfismo: sobrescritura (`@Override`), casting |

- | 15 | Clases abstractas vs interfaces |
- | 16 | Interfaces: implementación múltiple, `default` methods |
- | 17 | Modificadores: `static`, `final`, `this` |
- | 18 | Clases internas, anónimas y locales |
- | 19 | Enumeraciones (`enum`) y anotaciones básicas |
- | 20 | Paquetes, imports y gestión de dependencias con Maven |
- | 21 | Manejo de excepciones: `try-catch-finally`, `throws` |
- | 22 | Excepciones personalizadas, jerarquía de excepciones |
- | 23 | Proyecto práctico OOP: Sistema de Biblioteca |
- | 24 | UML básico: diagramas de clase aplicados a Java |
- | 25 | Revisión y ejercicios integradores de OOP |

BLOQUE 3: Colecciones y Genéricos (Clases 26–33)

Clase Tema
----- -----
26 `ArrayList`, `LinkedList`, diferencias y cuándo usar cada una
27 `HashMap`, `HashSet`, `LinkedHashMap`
28 `TreeMap`, `TreeSet`, ordenación natural
29 Iteradores, `for-each`, `Iterator`
30 Genéricos: ` <t>`, wildcards `<?>`, bounded types </td> </t>
31 `Collections` utility class: `sort`, `shuffle`, `binarySearch`
32 `Stack`, `Queue`, `Deque`, `PriorityQueue`
33 Proyecto práctico: Gestión de inventario con colecciones

BLOQUE 4: Java Funcional y Streams (Clases 34–40)

| Clase | Tema |

|-----|-----|

| 34 | Expresiones lambda: sintaxis, contexto funcional |

| 35 | Interfaces funcionales: ``Predicate``, ``Function``, ``Consumer``, ``Supplier`` |

| 36 | Stream API: ``filter``, ``map``, ``collect``, ``reduce`` |

| 37 | Streams avanzados: ``flatMap``, ``groupingBy``, ``partitioningBy`` |

| 38 | ``Optional<T>``: evitar ``NullPointerException`` |

| 39 | Referencia a métodos (``Method References``) |

| 40 | Proyecto práctico: Procesamiento de datos con Streams |

BLOQUE 5: I/O, Serialización y Concurrencia (Clases 41–48)

| Clase | Tema |

|-----|-----|

| 41 | Manejo de archivos: ``File``, ``Path``, ``Files`` (NIO.2) |

| 42 | Lectura/escritura: ``BufferedReader``, ``BufferedWriter``, ``FileInputStream`` |

| 43 | Serialización y deserialización de objetos |

| 44 | Manejo de JSON con ``Jackson`` o ``Gson`` |

| 45 | Hilos (``Thread``): creación, ciclo de vida |

| 46 | ``Runnable``, ``Callable``, ``ExecutorService``, ``Future`` |

| 47 | Sincronización: ``synchronized``, ``volatile``, ``ReentrantLock`` |

| 48 | ``CompletableFuture`` y programación asíncrona básica |

BLOQUE 6: Bases de Datos y APIs (Clases 49–56)

| Clase | Tema |

|-----|-----|

| 49 | JDBC: conexión a BD, `DriverManager`, `Connection` |

| 50 | CRUD con JDBC: `PreparedStatement`, `ResultSet` |

| 51 | Introducción a JPA / Hibernate: entidades, `EntityManager` |

| 52 | Relaciones JPA: `@OneToMany`, `@ManyToMany`, `@JoinColumn` |

| 53 | Spring Boot: configuración inicial, estructura del proyecto |

| 54 | REST API con Spring Boot: `@RestController`, `@GetMapping`,
`@PostMapping` |

| 55 | Spring Data JPA: repositorios, consultas JPQL |

| 56 | Proyecto práctico: API REST CRUD completa con Spring Boot |

BLOQUE 7: Temas Avanzados (Clases 57–60)

| Clase | Tema |

|-----|-----|

| 57 | Patrones de diseño: Singleton, Factory, Builder, Observer |

| 58 | Principios SOLID aplicados a Java |

| 59 | Herramientas del ecosistema: Maven, Git, Docker básico |

| 60 | Proyecto final: aplicación completa con Spring Boot, JPA, REST y manejo de
errores |

MÓDULO 2 — Pruebas Unitarias con JUnit (Aprox 20 clases × 1 hora)

| Clase | Tema |

|-----|-----|

- | 1 | Qué es una prueba unitaria, principios F.I.R.S.T, pirámide de testing |
- | 2 | Configuración de JUnit 5 con Maven/Gradle |
- | 3 | Anatomía de un test: `@Test`, `@BeforeEach`, `@AfterEach`, `@BeforeAll` |
- | 4 | Assertions: `assertEquals`, `assertTrue`, `assertThrows`, `assertAll` |
- | 5 | Ciclo de vida de tests y anotaciones de ciclo (`@Nested`) |
- | 6 | Tests parametrizados: `@ParameterizedTest`, `@ValueSource`, `@CsvSource` |
- | 7 | `@MethodSource`, `@EnumSource` y fuentes de datos externas |
- | 8 | Mocking con Mockito: `@Mock`, `@InjectMocks`, `when().thenReturn()` |
- | 9 | Verificación con Mockito: `verify()`, `times()`, `ArgumentCaptor` |
- | 10 | Spies, stubs y mocks: diferencias y cuándo usarlos |
- | 11 | TDD (Test Driven Development): ciclo Red-Green-Refactor |
- | 12 | Cobertura de código con JaCoCo |
- | 13 | Testing de excepciones y casos negativos |
- | 14 | Testing de clases con dependencias: inyección con `@ExtendWith` |
- | 15 | Testing de repositorios con H2 en memoria |
- | 16 | Testing de servicios Spring Boot con `@SpringBootTest` |
- | 17 | `@WebMvcTest` y `MockMvc`: testing de controladores REST |
- | 18 | Testing de componentes aislados con `@DataJpaTest` |
- | 19 | Organización de suites de tests, tags (`@Tag`), exclusiones |
- | 20 | Buenas prácticas: nombres de tests, AAA pattern, mantenibilidad |

MÓDULO 3 — Pruebas Unitarias con JUnit para Programas AS400 / IBM i (Aprox 15
clases × 1 hora)

| Clase | Tema |

|-----|-----|

- | 1 | Introducción al entorno IBM i / AS400: estructura, objetos, RPGLE |
- | 2 | Arquitectura de integración Java ↔ AS400: JT400 / IBM Toolbox for Java |
- | 3 | Configuración de dependencia `jt400.jar` en Maven |
- | 4 | Conexión a AS400 con `AS400` class, `SecureAS400`, autenticación |
- | 5 | Llamada a programas RPG con `ProgramCall` y `ProgramParameter` |
- | 6 | Llamada a comandos CL con `CommandCall` |
- | 7 | Estructuras de datos (Data Structures) y mapeo Java ↔ AS400 |
- | 8 | Lectura de archivos físicos con `KeyedFile`, `SequentialFile` |
- | 9 | Acceso a colas de datos (`DataQueue`) y colas de mensajes |
- | 10 | Estrategia de testing: mockear JT400 con Mockito |
- | 11 | Mock de `AS400`, `ProgramCall`, `ProgramParameter` en tests unitarios |
- | 12 | Testing de servicios que llaman programas RPG con datos de prueba |
- | 13 | Testing de lectura/escritura de archivos físicos (mocked) |
- | 14 | Integración con JUnit 5: suites de tests para módulos AS400 |
- | 15 | Proyecto práctico: suite de tests unitarios para módulo AS400 completo |

MÓDULO 4 — Pruebas de Integración con Karate (15 clases)

| Clase | Tema |

|-----|-----|

- | 1 | Qué es Karate: BDD + API testing, comparación con RestAssured |

2 Configuración del proyecto Karate con Maven
3 Estructura de un feature file: `Feature`, `Scenario`, `Given/When/Then`
4 Primeras pruebas HTTP: `GET`, `POST`, validación de `status` y `response`
5 Sintaxis Karate: variables, `def`, `match`, `contains`, `==`
6 JSON y XML: matching parcial, `#notnull`, `#array`, esquemas
7 `PUT`, `PATCH`, `DELETE`: pruebas de API REST completa
8 Parametrización con `Examples`: en Scenario Outline
9 Llamadas entre features: `call`, `callonce`, reutilización
10 Autenticación: Basic Auth, Bearer Token, OAuth2
11 Headers, cookies, multipart y subida de archivos
12 Configuración de entornos: `karate-config.js`, perfiles
13 Mocking de servicios con Karate Netty (`karate-mock`)
14 Reportes HTML con Karate Reports y integración con CI/CD
15 Proyecto práctico: suite de integración completa para una API REST

MÓDULO 5 — Pruebas E2E con Playwright + Serenity BDD (Aprox 20 clases)

Clase Tema
----- -----
1 Introducción a E2E testing: propósito, cuándo usarlo, herramientas
2 Playwright: instalación, arquitectura, soporte multi-browser
3 Primer test E2E con Playwright en Java (`playwright-java`)
4 Selectores: CSS, XPath, `getByRole`, `getByText`, `getByLabel`
5 Acciones: `click`, `fill`, `select`, `hover`, `press`, `dragAndDrop`

- | 6 | Assertions con ``expect()``: ``toBeVisible``, ``toHaveText``, ``toHaveURL`` |
- | 7 | Waits inteligentes: auto-wait de Playwright, ``waitForSelector``, timeouts |
- | 8 | Page Object Model (POM) con Playwright |
- | 9 | Manejo de ventanas, popups, iframes y file downloads |
- | 10 | Interceptación de red: ``route()``, mock de APIs en E2E |
- | 11 | Introducción a Serenity BDD: filosofía, reportes vivos |
- | 12 | Configuración de Serenity con JUnit 5 y Maven |
- | 13 | ``@Steps`` y ``StepLibrary``: pasos reutilizables con Serenity |
- | 14 | Integración Playwright + Serenity: ``SerenityRunner``, anotaciones |
- | 15 | Escritura de tests BDD con Cucumber + Serenity: ``feature`` files |
- | 16 | ``@Given``, ``@When``, ``@Then`` en Step Definitions con Serenity |
- | 17 | Screenshots automáticos, narrativas y reportes Serenity HTML |
- | 18 | Datos de prueba: tablas en Cucumber, Faker, fixtures |
- | 19 | Paralelización de tests E2E y configuración de CI/CD (GitHub Actions) |
- | 20 | Proyecto final: suite E2E completa con Playwright + Serenity + BDD |

NOTA: Cada módulo construye sobre el anterior. Se debe incluir todo un esquema de practica, talleres, mini proyectos, se puede completar en un aproximado de ****18 meses****.

MÓDULO 6 — DevOps Básico + GitHub Actions + Automatización de Pruebas (30 clases)

BLOQUE 1: Fundamentos de DevOps (Clases 1–6)

| Clase | Tema |

|-----|-----|

- | 1 | Qué es DevOps: cultura, principios, ciclo CI/CD, diferencia Dev vs Ops |
- | 2 | Control de versiones con Git: `clone`, `commit`, `branch`, `merge`, `rebase` |
- | 3 | Flujos de trabajo Git: GitFlow, trunk-based development, pull requests |
- | 4 | Introducción a contenedores: qué es Docker, imágenes vs contenedores |
- | 5 | Docker práctico: `Dockerfile`, `docker build`, `docker run`, `docker-compose`
|
- | 6 | Registros de imágenes: Docker Hub, GitHub Container Registry (`ghcr.io`) |

BLOQUE 2: GitHub Actions desde cero (Clases 7–14)

| Clase | Tema |

|-----|-----|

- | 7 | Qué es GitHub Actions: eventos, workflows, jobs, steps |
- | 8 | Anatomía de un workflow YAML: `on`, `jobs`, `runs-on`, `steps`, `uses` |
- | 9 | Eventos de disparo: `push`, `pull\_request`, `schedule`, `workflow\_dispatch` |
- | 10 | Actions del marketplace: `actions/checkout`, `actions/setup-java`,
`actions/cache` |
- | 11 | Variables de entorno, `env`, secretos con `secrets.\*` y `vars.\*` |
- | 12 | Jobs en paralelo y secuenciales: `needs`, matrices con `strategy.matrix` |
- | 13 | Artefactos: `upload-artifact`, `download-artifact`, retención |
- | 14 | Reusable workflows: `workflow\_call`, reutilización entre repositorios |

BLOQUE 3: Pipeline de Build y Calidad de Código (Clases 15–18)

| Clase | Tema |

|-----|-----|

| 15 | Pipeline de compilación Java con Maven: ``mvn compile``, ``package``, ``verify`` |

| 16 | Análisis estático de código: Checkstyle, SpotBugs, PMD en pipeline |

| 17 | Integración con SonarCloud: escaneo de calidad automático en PR |

| 18 | Caché de dependencias Maven/Gradle en GitHub Actions para acelerar builds |

BLOQUE 4: Ejecución de Pruebas Unitarias en Pipeline (Clases 19–21)

| Clase | Tema |

|-----|-----|

| 19 | Job de JUnit 5 en GitHub Actions: ``mvn test``, reporte de resultados |

| 20 | Publicar resultados de tests con ``dorny/test-reporter`` y ``junit-reporter`` |

| 21 | Cobertura de código con JaCoCo: generar reporte y publicar en PR como comentario |

BLOQUE 5: Ejecución de Pruebas de Integración con Karate (Clases 22–23)

| Clase | Tema |

|-----|-----|

| 22 | Levantar la API bajo prueba con Docker en el pipeline: `services` y `docker-compose` |

| 23 | Job de Karate en GitHub Actions: configurar entorno, ejecutar tests, publicar reporte HTML como artefacto |

BLOQUE 6: Ejecución de Pruebas E2E con Playwright + Serenity (Clases 24–26)

| Clase | Tema |

|-----|-----|

| 24 | Instalación de browsers en CI: `playwright install --with-deps` en pipeline |

| 25 | Job E2E headless con Playwright + JUnit en GitHub Actions |

| 26 | Publicar reporte Serenity HTML como artefacto y como GitHub Pages |

BLOQUE 7: Pipeline Completo y Buenas Prácticas (Clases 27–30)

| Clase | Tema |

|-----|-----|

| 27 | Pipeline multi-stage completo: build → unit tests → integration tests → E2E |

| 28 | Estrategia de ramas: bloquear merge si algún stage falla, branch protection rules |

| 29 | Notificaciones: Slack, email y comentarios automáticos en PR con resultados |

| 30 | Proyecto final: pipeline end-to-end desde commit hasta reporte publicado |

NOTA: Cada módulo construye sobre el anterior. Se debe incluir todo un esquema de practica, talleres, mini proyectos, se puede completar en un aproximado de **18 meses**.